WazuhBot — A Local-First, Agentic SOC Assistant for Wazuh SIEM

Muhammad Jawad Maqsood, Minahil Siddiq, Tooba Iqbal Aswani

SUPERVISOR: Dr. Faisal Iradat | **INSTITUTION:** SMCS, Institute of Business Administration, Karachi | **YEAR:** FYP 2025–2026

PROBLEM STATEMENT

SIEM Limitations: Traditional SIEM platforms (like Wazuh and QRadar) surface massive volumes of raw logs, fragmented alerts, and complex OpenSearch query results. However, they lack an intuitive, conversational interface to quickly piece together the full context of a threat. Analysts must manually pivot between dashboards, construct rigid database queries, and manually cross-reference rule IDs, drastically slowing down the investigation process.

SOC Analyst Fatigue: Security Operation Center (SOC) analysts are overwhelmed by alert fatigue. Manually correlating raw network flows, investigating thousands of false positives, and executing complex, repetitive API queries leads to cognitive overload, burnout, and critically delayed incident response times (MTTR). The sheer volume of noise makes it difficult to focus on actual, high-priority threats.

Privacy Risk of Cloud LLMs: While AI could accelerate triage, feeding sensitive security telemetry (such as internal IP addresses, agent configurations, user credentials, and network architecture details) into external, proprietary cloud AI models creates severe data egress vulnerabilities. This violates strict enterprise privacy policies and compliance frameworks, making cloud-based LLMs fundamentally unsuitable for secure SOC operations.

SOC analyst overwhelmed by alerts

Key Problem Summary

- Lack of a conversational, agentic interface for fast threat triage.
- High false-positive rate drives analyst fatigue and slow MTTR.
- Cloud LLMs leak sensitive telemetry — unacceptable in regulated industries.
- Static rule sets can't keep up with novel attack patterns.

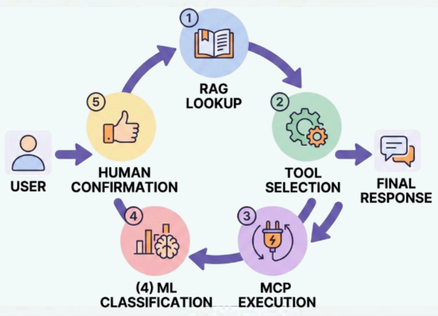
Requirements for a Local-First SOC Assistant

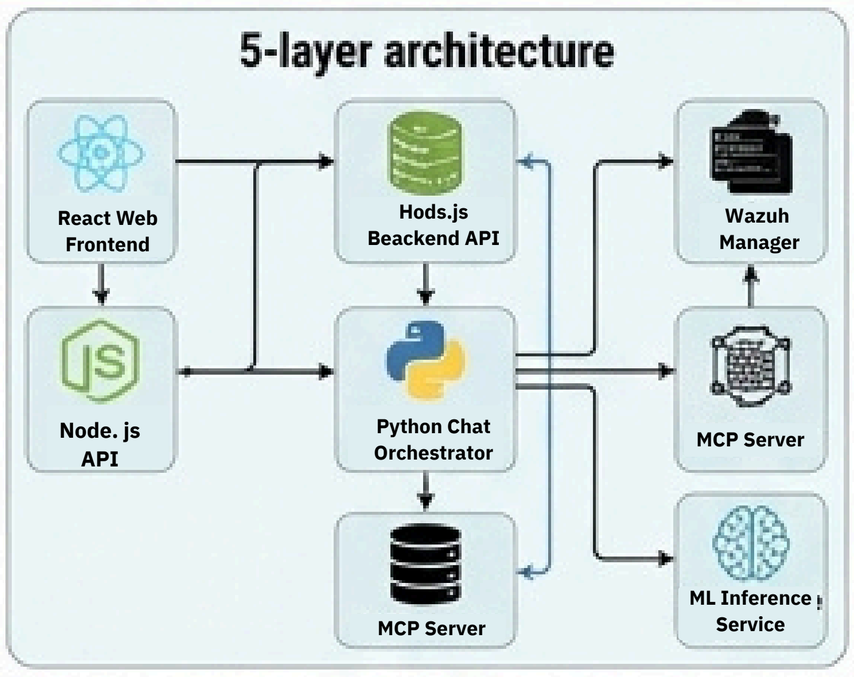
1. Locally-hosted, agentic LLM interface.
2. Automated ML threat classification.
3. Zero data egress.
4. Explicit human remediation safeguard

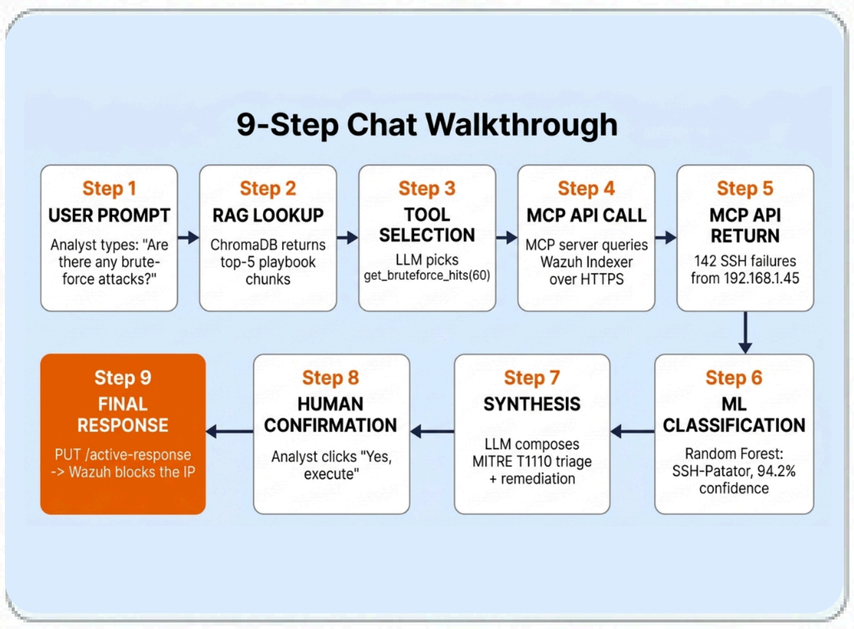
SOLUTION SECTION: Architecture & Methods

5-Process Distributed Architecture: WazuhBot runs on a highly modular, scalable system consisting of a React Web Frontend, a secure Node.js Backend API, a central Python Chat Orchestrator (chatbot_server.py), an isolated MCP Server (universal_api_server.py) for bridging Wazuh API commands, and a dedicated ML Inference Service (inference_server.py). This separation of concerns ensures stability, security, and independent scalability.

Agentic RAG + MCP Tool-Calling Loop: Instead of merely answering questions, the Python orchestrator runs an autonomous multi-step evaluation loop. It first fetches domain-specific local knowledge via Retrieval-Augmented Generation (using a ChromaDB vector store). The LLM then dynamically interacts with live Wazuh APIs using the Model Context Protocol (MCP)—autonomously deciding which pre-built tools to call (e.g., fetching alerts, querying agent status, or running raw OpenSearch queries) to investigate events in real-time before synthesizing a final response.







Random Forest Threat Classifier: A local machine learning service processes raw network flow data from Wazuh alerts, automatically enriching 9 basic fields into 40 distinct features (e.g., average throughput, TCP flags, duration). It then passes these features through a pre-trained scikit-learn Random Forest model to instantly classify anomalies—such as Command & Control (C2) beacons, Port Scans, and Brute Force attacks—providing the LLM with precise confidence scoring and threat probabilities.

confirm_write=True Safety Gate: To prevent AI hallucinations from causing system damage or unauthorized configuration changes, the universal API server enforces a strict, hardcoded security boundary. It automatically blocks any state-modifying HTTP actions (like PUT, POST, or DELETE for restarting agents or dropping firewall rules) unless the human analyst explicitly grants permission by setting a confirm_write=True flag, ensuring a strict "human-in-the-loop" safeguard for all remediation actions.

Local-First / Zero Data Egress: Powered entirely by on-premise infrastructure, WazuhBot utilizes locally hosted Small Language Models (via Ollama, e.g., qwen2.5:3b) and local vector embeddings. This design guarantees zero data egress—ensuring that all sensitive SIEM telemetry, chat histories, and investigation contexts remain securely contained within the organization's private network, fully compliant with data privacy regulations.

KEY METRICS

~3–8 s end-to-end chat latency · **>89.8%** ML accuracy · **$0** infra cost · **0 bytes** telemetry leaving the org